LABSHEET 3

REBUILDING THE SITE IN REACT (JAVASCRIPT LIBRARY)

The SALT Beauty store from Labsheet 2 (plain HTML/CSS/JS with a localStorage-based cart) is rebuilt below as a React single-page app. The four pages (Home, Products, Cart, Checkout) become components swapped by state instead of separate HTML files, and the cart moves out of localStorage into a shared CartContext so every component reads from one source of truth.

INDEX.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SALT | Beauty</title>
  <link rel="stylesheet" href="dist/bundle.css" />
</head>
<body>
  <div id="root"></div>
  <script src="dist/bundle.js"></script>
</body>
</html>
```

SRC/MAIN.JSX

```
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")).render(<App />);
```

SRC/APP.JSX

```
import { useState } from "react";
import { CartProvider } from "./context/CartContext";
import Header from "./components/Header";
import Footer from "./components/Footer";
import Toast from "./components/Toast";
import Home from "./pages/Home";
import Products from "./pages/Products";
import Cart from "./pages/Cart";
import Checkout from "./pages/Checkout";

// The static site used four HTML files (index/products/cart/checkout.html).
// In React that becomes one page component swapped by a bit of state,
// which is the simplest form of client-side routing without pulling in
// react-router for a four-page site.
export default function App() {
  const [page, setPage] = useState("home");

  function renderPage() {
    switch (page) {
      case "products":
        return <Products />;
      case "cart":
        return <Cart setPage={setPage} />;
      case "checkout":
        return <Checkout />;
```

```
      default:
        return <Home setPage={setPage} />;
    }
  }

  return (
    <CartProvider>
      <Header page={page} setPage={setPage} />
      {renderPage()}
      <Footer />
      <Toast />
    </CartProvider>
  );
}
```

SRC/CONTEXT/CARTCONTEXT.JSX

```
import { createContext, useContext, useMemo, useState } from "react";

// In the vanilla JS version the cart lived in localStorage and every
// page re-read it with getCart(). In React we lift that state up once
// and share it through Context, so every component that needs the
// cart (Header badge, Cart page, Checkout page) reads from the same
// single source of truth instead of touching the DOM/storage directly.
const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [cart, setCart] = useState([]);
  const [lastMessage, setLastMessage] = useState("");

  function addToCart(product, quantity = 1) {
    setCart((prev) => {
      const existing = prev.find((item) => item.id === product.id);
      if (existing) {
        return prev.map((item) =>
          item.id === product.id
            ? { ...item, quantity: item.quantity + quantity }
            : item
        );
      }
      return [...prev, { ...product, quantity }];
    });
    setLastMessage("Product added to cart!");
  }

  function changeQuantity(id, change) {
    setCart((prev) =>
      prev
        .map((item) =>
          item.id === id ? { ...item, quantity: item.quantity + change } : item
        )
        .filter((item) => item.quantity > 0)
    );
  }

  function removeItem(id) {
    setCart((prev) => prev.filter((item) => item.id !== id));
  }

  function clearCart() {
    setCart([]);
  }

  const count = useMemo(
    () => cart.reduce((sum, item) => sum + item.quantity, 0),
    [cart]
  );

  const total = useMemo(
```

```
    () => cart.reduce((sum, item) => sum + item.price * item.quantity, 0),
    [cart]
  );

  const value = {
    cart,
    count,
    total,
    addToCart,
    changeQuantity,
    removeItem,
    clearCart,
    lastMessage,
    setLastMessage,
  };

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used inside a CartProvider");
  return ctx;
}
```

SRC/DATA/PRODUCTS.JS

```
// Product catalogue for the SALT beauty store.
// Each product points at its photo in /public/images (cropped from the
// product shots used in the original Labsheet 2 site).
const products = [
  {
    id: 1,
    name: "Hydrating Face Serum",
    price: 899,
    image: "images/serum.jpg",
  },
  {
    id: 2,
    name: "Rose Glow Cream",
    price: 749,
    image: "images/moisturizer.jpg",
  },
  {
    id: 3,
    name: "Gentle Daily Cleanser",
    price: 599,
    image: "images/cleanser.jpg",
  },
  {
    id: 4,
    name: "Nourishing Face Oil",
    price: 999,
    image: "images/face-oil.jpg",
  },
  {
    id: 5,
    name: "Cloud Body Butter",
    price: 699,
    image: "images/body-butter.jpg",
  },
  {
    id: 6,
    name: "Rose Clay Mask",
    price: 649,
    image: "images/clay-mask.jpg",
  },
  {
    id: 7,
```

```
    name: "Soft Glow Lip Oil",
    price: 449,
    image: "images/lip-oil.jpg",
  },
  {
    id: 8,
    name: "Overnight Glow Cream",
    price: 849,
    image: "images/night-cream.jpg",
  },
];

export default products;
```

SRC/COMPONENTS/HEADER.JSX

```
import { useCart } from "../context/CartContext";

export default function Header({ page, setPage }) {
  const { count } = useCart();

  function go(target) {
    return (e) => {
      e.preventDefault();
      setPage(target);
    };
  }

  return (
    <header>
      <nav>
        <a href="#home" className="logo" onClick={go("home")}>
          SALT
        </a>
        <div className="nav-links">
          <a
            href="#home"
            className={page === "home" ? "active" : ""}
            onClick={go("home")}
          >
            Home
          </a>
          <a
            href="#products"
            className={page === "products" ? "active" : ""}
            onClick={go("products")}
          >
            Products
          </a>
          <a
            href="#cart"
            className={page === "cart" ? "active" : ""}
            onClick={go("cart")}
          >
            Cart (<span id="cart-count">{count}</span>)
          </a>
        </div>
      </nav>
    </header>
  );
}
```

SRC/COMPONENTS/FOOTER.JSX

```
export default function Footer() {
  return (
    <footer>
      <h2>SALT</h2>
      <p>hello@saltbeauty.com</p>
```

```
      <p>&copy; 2026 SALT Beauty</p>
    </footer>
  );
}
```

SRC/COMPONENTS/PRODUCTCARD.JSX

```
import ProductImage from "./ProductImage";
import { useCart } from "../context/CartContext";

export default function ProductCard({ product }) {
  const { addToCart } = useCart();

  return (
    <div className="card">
      <ProductImage image={product.image} name={product.name} />
      <h3>{product.name}</h3>
      <p>&#8377;{product.price}</p>
      <button className="btn" onClick={() => addToCart(product)}>
        Add to Cart
      </button>
    </div>
  );
}
```

SRC/COMPONENTS/PRODUCTIMAGE.JSX

```
// Renders a product photo. Falls back to a soft gradient tile if an
// image is ever missing, so the layout never breaks.
export default function ProductImage({ image, name }) {
  if (!image) {
    return (
      <div className="product-image product-image-fallback" role="img" aria-label={name} />
    );
  }
  return (
    <img className="product-image" src={image} alt={name} loading="lazy" />
  );
}
```

SRC/COMPONENTS/TOAST.JSX

```
import { useEffect } from "react";
import { useCart } from "../context/CartContext";

// The vanilla JS version used window.alert("Product added to cart!").
// A blocking alert() is poor UX and awkward to test in React, so this
// swaps it for a small self-dismissing toast driven by cart state.
export default function Toast() {
  const { lastMessage, setLastMessage } = useCart();

  useEffect(() => {
    if (!lastMessage) return;
    const timer = setTimeout(() => setLastMessage(""), 1800);
    return () => clearTimeout(timer);
  }, [lastMessage, setLastMessage]);

  if (!lastMessage) return null;

  return <div className="toast">{lastMessage}</div>;
}
```

SRC/PAGES/HOME.JSX

```
import products from "../data/products";
import ProductCard from "../components/ProductCard";

export default function Home({ setPage }) {
```

```
  const featured = products.slice(0, 4);

  return (
    <>
      <section className="hero">
        <div>
          <p>BEAUTY, SIMPLIFIED.</p>
          <h1>
            Simple beauty.
            <br />
            <i>Real results.</i>
          </h1>
          <p>Everyday skincare made simple, gentle and beautiful.</p>
          <a
            href="#products"
            className="btn"
            onClick={(e) => {
              e.preventDefault();
              setPage("products");
            }}
          >
            Shop Collection
          </a>
        </div>
      </section>

      <section className="products">
        <p>OUR FAVOURITES</p>
        <h2>Featured Products</h2>
        <div className="grid">
          {featured.map((product) => (
            <ProductCard key={product.id} product={product} />
          ))}
        </div>
      </section>
    </>
  );
}
```

SRC/PAGES/PRODUCTS.JSX

```
import products from "../data/products";
import ProductCard from "../components/ProductCard";

export default function Products() {
  return (
    <section className="products products-page">
      <p>THE SALT COLLECTION</p>
      <h2>All Products</h2>
      <div className="grid">
        {products.map((product) => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </section>
  );
}
```

SRC/PAGES/CART.JSX

```
import ProductImage from "../components/ProductImage";
import { useCart } from "../context/CartContext";

export default function Cart({ setPage }) {
  const { cart, total, changeQuantity, removeItem } = useCart();

  return (
    <section className="cart-page">
      <p>YOUR BAG</p>
```

```
      <h2>Shopping Cart</h2>

      {cart.length === 0 ? (
        <p className="empty">Your cart is empty.</p>
      ) : (
        <div className="cart-rows">
          {cart.map((item) => (
            <div className="cart-row" key={item.id}>
              <ProductImage image={item.image} name={item.name} />
              <div className="cart-info">
                <h3>{item.name}</h3>
                <p>&#8377;{item.price}</p>
                <div className="quantity">
                  <button onClick={() => changeQuantity(item.id, -1)}>&minus;</button>
                  <span>{item.quantity}</span>
                  <button onClick={() => changeQuantity(item.id, 1)}>+</button>
                </div>
              </div>
              <div className="cart-row-end">
                <strong>&#8377;{item.price * item.quantity}</strong>
                <br />
                <button className="remove" onClick={() => removeItem(item.id)}>
                  Remove
                </button>
              </div>
            </div>
          ))}
        </div>
      )}

      <h3 className="grand-total">Grand Total: &#8377;{total}</h3>

      <button
        className="btn"
        disabled={cart.length === 0}
        onClick={() => setPage("checkout")}
      >
        Proceed to Checkout
      </button>
    </section>
  );
}
```

SRC/PAGES/CHECKOUT.JSX

```
import { useState } from "react";
import { useCart } from "../context/CartContext";

const initialForm = {
  fullName: "",
  address: "",
  city: "",
  pincode: "",
  phone: "",
  payment: "",
};

export default function Checkout() {
  const { cart, total, clearCart } = useCart();
  const [form, setForm] = useState(initialForm);
  const [message, setMessage] = useState("");
  const [placed, setPlaced] = useState(false);

  function handleChange(e) {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  }

  function handleSubmit(e) {
```

```
  e.preventDefault();

  if (cart.length === 0) {
    setMessage("Your cart is empty.");
    return;
  }
  if (!form.payment) {
    setMessage("Please select a payment method.");
    return;
  }

  setMessage("Order placed successfully! Thank you for shopping with SALT.");
  setPlaced(true);
  clearCart();
  setForm(initialForm);
}

return (
  <section className="checkout-page">
    <h2>Checkout</h2>

    <div className="checkout-grid">
      <div className="order-summary">
        <h3>Your Order</h3>
        {placed ? (
          <p>Order completed.</p>
        ) : cart.length === 0 ? (
          <p>Your cart is empty.</p>
        ) : (
          <div className="checkout-items">
            {cart.map((item) => (
              <div className="checkout-item" key={item.id}>
                <span>
                  {item.name} &times; {item.quantity}
                </span>
                <strong>&#8377;{item.price * item.quantity}</strong>
              </div>
            ))}
          </div>
        )}
        <p className="checkout-total">Total: &#8377;{placed ? 0 : total}</p>
      </div>

      <form className="checkout-form" onSubmit={handleSubmit}>
        <label>
          Full Name
          <input
            name="fullName"
            value={form.fullName}
            onChange={handleChange}
            placeholder="Enter your name"
          />
        </label>

        <label>
          Address
          <textarea
            name="address"
            value={form.address}
            onChange={handleChange}
            placeholder="Enter your address"
          />
        </label>

        <label>
          City
          <input
            name="city"
            value={form.city}
            onChange={handleChange}
```

```
              placeholder="Enter your city"
            />
          </label>

          <label>
            Pincode
            <input
              name="pincode"
              value={form.pincode}
              onChange={handleChange}
              placeholder="Enter your pincode"
            />
          </label>

          <label>
            Phone
            <input
              name="phone"
              value={form.phone}
              onChange={handleChange}
              placeholder="Enter your phone number"
            />
          </label>

          <fieldset className="payment-method">
            <legend>Payment Method</legend>
            {["UPI", "Card", "Cash on Delivery"].map((option) => (
              <label className="radio" key={option}>
                <input
                  type="radio"
                  name="payment"
                  value={option}
                  checked={form.payment === option}
                  onChange={handleChange}
                />
                {option}
              </label>
            ))}
          </fieldset>

          <button type="submit" className="btn">
            Place Order
          </button>

          {message && <p className="order-message">{message}</p>}
        </form>
      </div>
    </section>
  );
}
```

SRC/INDEX.CSS

```
:root {
  --font-display: Georgia, "Iowan Old Style", "Palatino Linotype", "Book Antiqua", serif;
  --font-body: -apple-system, BlinkMacSystemFont, "Segoe UI", Helvetica, Arial, sans-serif;
  --cream: #fbf3ea;
  --cream-deep: #f5e8da;
  --ink: #1f1b17;
  --rose: #d9a3ab;
  --line: rgba(31, 27, 23, 0.12);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
```

```
  background: var(--cream);
  color: var(--ink);
  font-family: var(--font-body);
}

a {
  color: inherit;
  text-decoration: none;
}

/* ---------- Header ---------- */
header {
  background: var(--cream);
  border-bottom: 1px solid var(--line);
  position: sticky;
  top: 0;
  z-index: 10;
}

nav {
  max-width: 1100px;
  margin: 0 auto;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 18px 32px;
}

.logo {
  font-family: var(--font-display);
  font-size: 22px;
  letter-spacing: 0.14em;
  font-weight: 500;
}

.nav-links {
  display: flex;
  gap: 28px;
  font-size: 14px;
}

.nav-links a {
  padding-bottom: 4px;
  border-bottom: 1px solid transparent;
}

.nav-links a.active,
.nav-links a:hover {
  border-bottom: 1px solid var(--ink);
}

/* ---------- Buttons ---------- */
.btn {
  display: inline-block;
  background: var(--ink);
  color: var(--cream);
  border: none;
  padding: 12px 26px;
  font-size: 13px;
  letter-spacing: 0.04em;
  cursor: pointer;
  border-radius: 2px;
}

.btn:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}

.btn:hover:not(:disabled) {
```

```
  background: #3a332c;
}

/* ---------- Hero ---------- */
.hero {
  max-width: 1100px;
  margin: 0 auto;
  padding: 90px 32px 70px;
}

.hero > p {
  letter-spacing: 0.18em;
  font-size: 12px;
  color: #8a7f74;
  margin-bottom: 18px;
}

.hero h1 {
  font-family: var(--font-display);
  font-weight: 500;
  font-size: 54px;
  line-height: 1.12;
  margin: 0 0 22px;
  max-width: 11ch;
}

.hero h1 i {
  color: var(--rose);
}

.hero > div > p:nth-of-type(2) {
  max-width: 34ch;
  color: #5c534b;
  line-height: 1.6;
  margin-bottom: 30px;
}

/* ---------- Product grid ---------- */
.products {
  max-width: 1100px;
  margin: 0 auto;
  padding: 20px 32px 80px;
}

.products > p {
  letter-spacing: 0.18em;
  font-size: 12px;
  color: #8a7f74;
  margin-bottom: 8px;
}

.products h2 {
  font-family: var(--font-display);
  font-weight: 500;
  font-size: 30px;
  margin: 0 0 36px;
}

.grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 24px;
}

.card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 6px;
  padding: 16px;
  text-align: left;
```

```
}

.product-image {
  width: 100%;
  aspect-ratio: 1 / 1;
  border-radius: 4px;
  display: block;
  object-fit: cover;
  margin-bottom: 14px;
  background: var(--cream-deep);
}

.product-image-fallback {
  background: linear-gradient(160deg, var(--cream-deep), var(--rose));
}

.card h3 {
  font-size: 14px;
  font-weight: 600;
  margin: 0 0 4px;
}

.card > p {
  color: #8a7f74;
  font-size: 13px;
  margin: 0 0 12px;
}

.card .btn {
  width: 100%;
  text-align: center;
  padding: 10px;
}

/* ---------- Cart page ---------- */
.cart-page,
.checkout-page {
  max-width: 1100px;
  margin: 0 auto;
  padding: 50px 32px 90px;
  min-height: 50vh;
}

.cart-page > p:first-child {
  letter-spacing: 0.18em;
  font-size: 12px;
  color: #8a7f74;
  margin-bottom: 8px;
}

.cart-page h2 {
  font-family: var(--font-display);
  font-size: 30px;
  font-weight: 500;
  margin: 0 0 34px;
}

.cart-rows {
  display: flex;
  flex-direction: column;
  gap: 22px;
  margin-bottom: 30px;
}

.cart-row {
  display: grid;
  grid-template-columns: 72px 1fr auto;
  gap: 18px;
  align-items: center;
  border-bottom: 1px solid var(--line);
```

```
  padding-bottom: 20px;
}

.cart-row .product-image {
  width: 72px;
  height: 72px;
  margin-bottom: 0;
}

.cart-info h3 {
  margin: 0 0 4px;
  font-size: 15px;
}

.cart-info > p {
  margin: 0 0 8px;
  color: #8a7f74;
  font-size: 13px;
}

.quantity {
  display: flex;
  align-items: center;
  gap: 10px;
}

.quantity button {
  width: 26px;
  height: 26px;
  border: 1px solid var(--line);
  background: #fff;
  border-radius: 50%;
  cursor: pointer;
  font-size: 14px;
  line-height: 1;
}

.cart-row-end {
  text-align: right;
}

.remove {
  border: none;
  background: none;
  color: #b45b5b;
  font-size: 12px;
  cursor: pointer;
  padding: 0;
  margin-top: 6px;
}

.grand-total {
  font-size: 18px;
  margin: 10px 0 26px;
}

.empty {
  color: #8a7f74;
}

/* ---------- Checkout page ---------- */
.checkout-page h2 {
  font-family: var(--font-display);
  font-size: 30px;
  font-weight: 500;
  margin: 0 0 34px;
}

.checkout-grid {
  display: grid;
```

```
  grid-template-columns: 320px 1fr;
  gap: 40px;
  align-items: start;
}

.order-summary {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 6px;
  padding: 22px;
}

.order-summary h3 {
  margin-top: 0;
  font-size: 15px;
}

.checkout-item {
  display: flex;
  justify-content: space-between;
  font-size: 13px;
  margin-bottom: 10px;
}

.checkout-total {
  font-weight: 600;
  border-top: 1px solid var(--line);
  padding-top: 12px;
  margin-top: 12px;
}

.checkout-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  max-width: 420px;
}

.checkout-form label {
  display: flex;
  flex-direction: column;
  gap: 6px;
  font-size: 13px;
  font-weight: 500;
}

.checkout-form input,
.checkout-form textarea {
  border: 1px solid var(--line);
  border-radius: 4px;
  padding: 10px 12px;
  font-size: 13px;
  font-family: inherit;
  background: #fff;
}

.checkout-form textarea {
  resize: vertical;
  min-height: 60px;
}

.payment-method {
  border: none;
  padding: 0;
  margin: 0;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.payment-method legend {
```

```
  font-size: 13px;
  font-weight: 500;
  margin-bottom: 6px;
  padding: 0;
}

.radio {
  flex-direction: row !important;
  align-items: center;
  gap: 8px !important;
  font-weight: 400 !important;
}

.order-message {
  font-size: 13px;
  color: #4d7358;
}

/* ---------- Footer ---------- */
footer {
  background: var(--ink);
  color: var(--cream);
  padding: 40px 32px;
}

footer > div,
footer {
  max-width: 1100px;
  margin: 0 auto;
}

footer h2 {
  font-family: var(--font-display);
  font-weight: 500;
  letter-spacing: 0.1em;
  margin: 0 0 10px;
}

footer p {
  margin: 4px 0;
  font-size: 13px;
  color: #cfc6ba;
}

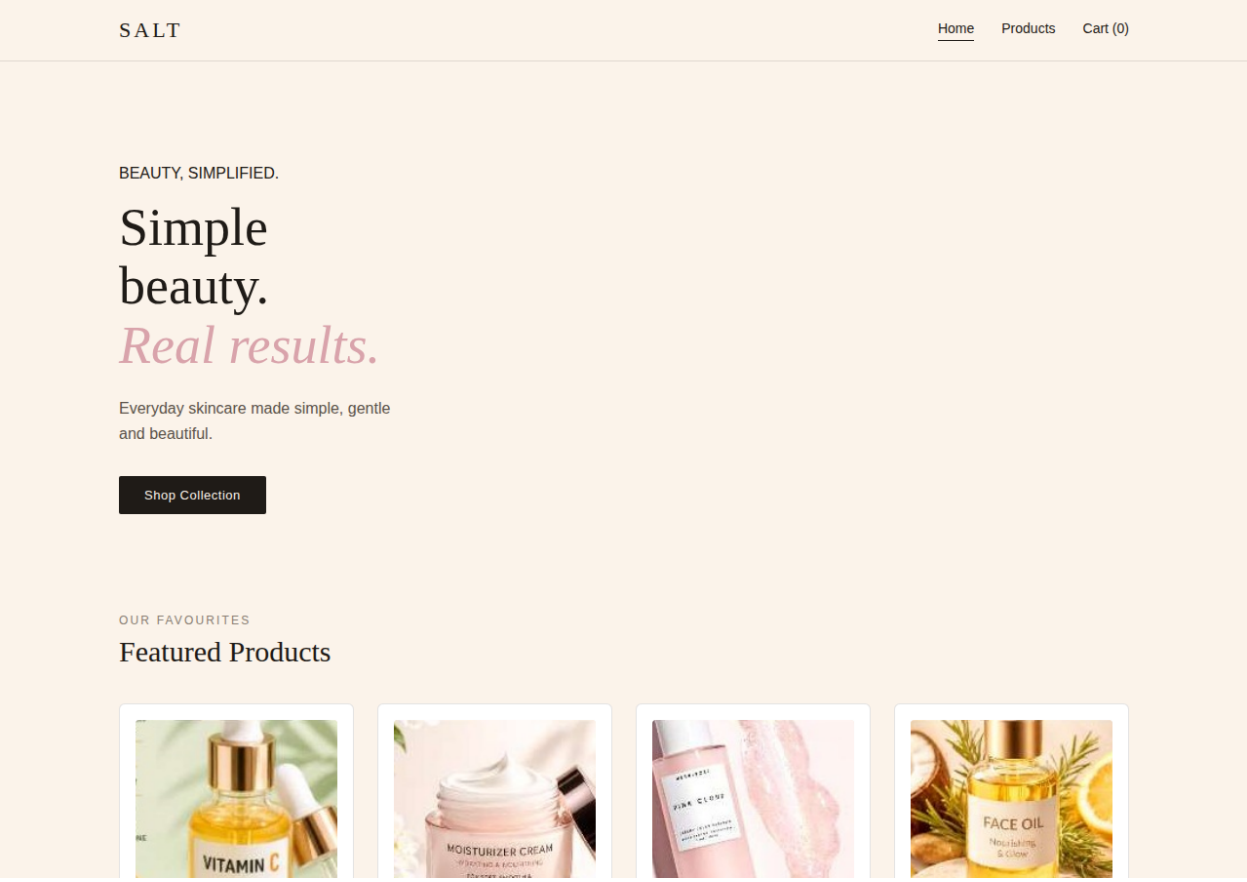
/* ---------- Toast ---------- */
.toast {
  position: fixed;
  bottom: 26px;
  left: 50%;
  transform: translateX(-50%);
  background: var(--ink);
  color: var(--cream);
  padding: 12px 22px;
  border-radius: 4px;
  font-size: 13px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.2);
}

@media (max-width: 720px) {
  .grid {
    grid-template-columns: repeat(2, 1fr);
  }
  .checkout-grid {
    grid-template-columns: 1fr;
  }
  .hero h1 {
    font-size: 38px;
  }
}
```

OUTPUT

Screenshots below were captured from the built React app running locally (Chromium via Playwright), walking through the same flow demonstrated in Labsheet 2: browse products, add to cart, view the bag, and place an order at checkout.

Home page



All Products page

SALT

Home Products Cart (0)

THE SALT COLLECTION

All Products



Hydrating Face Serum
₹899
Add to Cart



Rose Glow Cream
₹749
Add to Cart



Gentle Daily Cleanser
₹599
Add to Cart



Nourishing Face Oil
₹999
Add to Cart



Cloud Body Butter
₹699
Add to Cart



Rose Clay Mask
₹649
Add to Cart

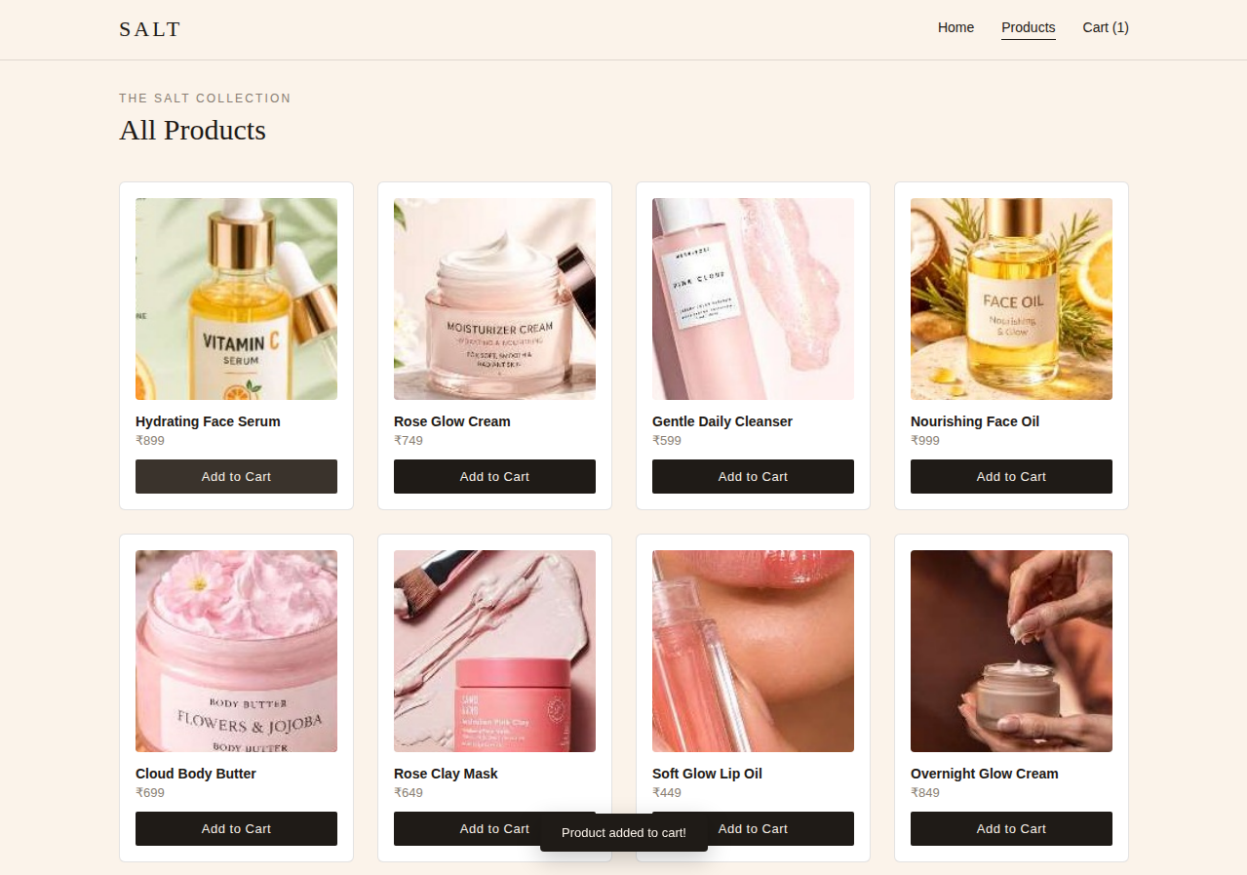


Soft Glow Lip Oil
₹449
Add to Cart



Overnight Glow Cream
₹849
Add to Cart

Adding a product to the cart (toast confirmation)



Shopping cart with items and grand total

SALT

Home Products Cart (2)

YOUR BAG

Shopping Cart



Hydrating Face Serum

₹899

− 1 +

₹899

Remove



Nourishing Face Oil

₹999

− 1 +

₹999

Remove

Grand Total: ₹1898

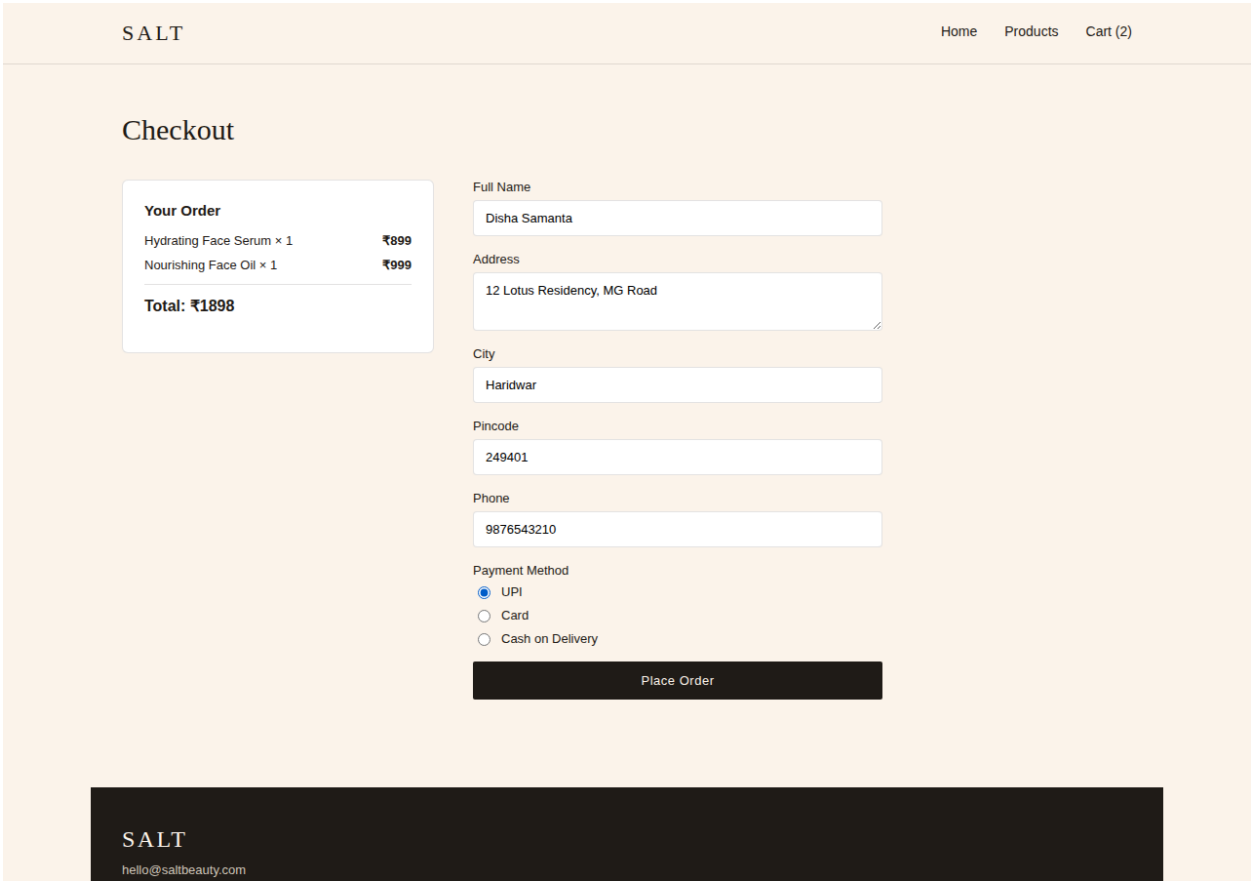
Proceed to Checkout

SALT

hello@saltbeauty.com

© 2026 SALT Beauty

Checkout form filled in with a payment method selected



Order placed successfully

